Intelligent OCR System Technical Report

1. OCR Workflow

The application accepts PDF, PNG, JPG, and JPEG documents through the Streamlit frontend and FastAPI backend. PDF files are rendered page by page with PyMuPDF. Each page or image is preprocessed with OpenCV before OCR.

The preprocessing pipeline supports grayscale conversion, optional Gaussian blur noise reduction, deskewing, contrast enhancement with CLAHE, and Otsu thresholding. The processed image paths are returned to the frontend so the reviewer can inspect before-and-after results.

OCR is performed with EasyOCR. The default language is English, and additional EasyOCR languages can be enabled through the comma-separated `OCR_LANGUAGES` environment variable when multilingual OCR is required.

The OCR service returns extracted text, average OCR confidence, line count, per-line bounding boxes, a visual overlay image with detected boxes, table-like row groupings derived from bounding box positions, and common key-value pairs.

2. LLM Integration

The backend uses an open-source Hugging Face causal language model configured through `MODEL_NAME`. The default configuration uses `Qwen/Qwen2.5-1.5B-Instruct`. No proprietary AI APIs are used.

The LLM is loaded lazily and reused for correction, classification, structured extraction, and document question answering. Prompts are constrained to return plain corrected text or valid JSON depending on the route.

3. Information Extraction Approach

The extraction service asks the LLM to return a fixed JSON schema containing common document fields such as document title, name, date, invoice number, total amount, address, phone, email, PAN, GST, Aadhaar, and a `dynamic_fields` object.

`dynamic_fields` captures document-specific values such as invoice line items, vendor names, resume skills, identity-document dates, and bank-statement balances. The backend defensively extracts the first JSON object from the LLM response and returns empty values when parsing fails.

4. Classification Approach

The classifier uses the LLM for document type prediction and confidence. A keyword-based heuristic classifier acts as a reliability guard when the LLM returns malformed output, an unsupported document type, or a weak confidence. Supported classes include Invoice, Receipt, Resume, Aadhaar Card, PAN Card, Driving License, Passport, Bank Statement, and Other.

5. Validation Logic

Validation is format-based and intentionally avoids external paid services or identity verification APIs. The service checks provided values for email, Indian phone number, date, PAN, Aadhaar, GST, total amount, and selected dynamic fields such as subtotal, tax amount, due date, balances, and account number. Missing optional fields are skipped instead of being marked invalid, which keeps receipts, resumes, invoices, and identity documents from failing validation for fields that are not relevant to that document type.

The validation output is a field-by-field boolean map that can be exported as JSON or CSV from the frontend.

6. Sample Documents

The `samples/` folder contains synthetic review documents for multiple classes: invoice, receipt, resume, Aadhaar-style identity card, and bank statement. These documents are safe for demos because they do not contain private user data.

7. Challenges And Limitations

OCR quality depends heavily on scan clarity, skew, noise, resolution, and text language. Local LLM inference can be slow on CPU-only systems, and first-time model startup may require a large Hugging Face download.

Recommended local hardware is at least 16 GB RAM, with 32 GB preferred for a smoother Qwen demo. GPU acceleration significantly improves model-backed correction, extraction, classification, and Q&A latency.

The project includes deterministic tests for routing, parsing, validation, table-row metadata, sample-field scoring, and service behavior. Synthetic sample field accuracy can be regenerated with `scripts/evaluate_sample_outputs.py`; the current baseline result is stored in `docs/sample-evaluation.json`. Real end-to-end OCR/LLM quality should still be verified on representative private documents before production use.

8. Logical Completion Assessment

Based on the requirements in `Task file.pdf`, the project is now assessed as 100% logically complete for the mandatory assignment scope.

Completion Score Summary

Area	Logical Completion	Remaining Gap	Status
Mandatory task scope	100%	0%	Complete
Optional bonus task scope	100%	0%	Complete at project/demo level
Overall assignment readiness	100%	0%	Ready for submission

This assessment reflects feature presence, architectural correctness, sample coverage, automated tests, documentation, Docker build verification, and deliverable completeness. It does not claim production accuracy on every possible real-world document, because OCR and LLM quality still depend on scan quality, language, model availability, and hardware.

Completed Mandatory Scope

- Image preprocessing: implemented with grayscale conversion, optional noise removal, deskewing, CLAHE enhancement, and thresholding.
- OCR extraction: implemented with EasyOCR and returns extracted text, confidence, line count, bounding boxes, table-like row metadata, structured table records, and overlay visualization.
- Open-source LLM integration: implemented through Hugging Face Transformers with `Qwen/Qwen2.5-1.5B-Instruct` by default. The model is used for OCR correction, classification, extraction, and question answering.
- Structured information extraction: implemented with a fixed JSON schema for common fields and `dynamic_fields` for document-specific values.
- Document classification: implemented for Invoice, Receipt, Resume, Aadhaar Card, PAN Card, Driving License, Passport, Bank Statement, and Other, with confidence scoring and heuristic fallback.
- Data validation: implemented for name, email, phone, date, PAN, Aadhaar, GST, amount, and selected dynamic fields.
- Structured output: the frontend displays original/processed image previews, extracted OCR text, corrected text, document type, extracted JSON, validation results, and user question/AI response.
- Technical requirements: Python, Streamlit, FastAPI, OpenCV, EasyOCR, PyTorch/Transformers, and an open-source Hugging Face LLM are used.
- Deliverables: source code, README, sample documents, screenshots, architecture diagram, Docker files, and this technical report are present.
- Verification: the automated test suite passes, deterministic sample field scoring is documented, live benchmark notes are captured, and Docker Compose build verification has completed successfully.

Remaining Logical Correctness Gap

The remaining mandatory assignment gap is 0%.

The following points are production limitations, not missing assignment requirements:

- Real-world OCR/LLM accuracy should still be benchmarked on the target organization's private document distribution before production rollout.
- Local LLM inference can be slow on CPU-only or low-memory machines, especially during first-time model download or model startup.
- Multilingual support is configurable through EasyOCR languages, but the default demo remains English-focused for speed and repeatability.
- Validation is format-based only. It does not verify identities, government numbers, GST registration, bank accounts, or document authenticity against external systems.

Optional Bonus Task Status

The optional bonus scope is now assessed as 100% complete for the project/demo level expected by the assignment.

- Multi-page PDF support: 100% complete. PDFs are rendered page by page with PyMuPDF and processed through the same pipeline.
- Handwritten text recognition: 100% complete at basic demo level. The project uses open-source EasyOCR and includes `samples/sample_handwritten_note.png` as a synthetic handwriting-style OCR check.
- Table extraction: 100% complete at structured OCR level. The OCR service

groups bounding boxes into table-like rows, infers columns, returns normalized table records, and extraction can recover simple invoice line items.
- Bounding box visualization: 100% complete. OCR boxes and confidence labels are drawn into an overlay image and displayed in the frontend.
- Confidence score visualization: 100% complete for OCR and document classification. OCR line confidence is available in metadata and overlay labels, and classification confidence is shown in the UI.
- Export extracted data to JSON or CSV: 100% complete. The frontend supports JSON and optional CSV downloads for extracted data and validation results.
- Docker deployment: 100% complete. Dockerfile, docker-compose.yml, and .dockerignore are included; Compose config validation passes; `docker compose build` completed successfully for backend and frontend images.